

TP Jour 2 — Modélisation, Indexation & Drivers (4 h)

Module MIA4 · Conception et intégration d'un SGBD NoSQL — IPSSI Montpellier Jour 2 / 5 — Schema design (embed vs reference), patterns, index & `explain()`, agrégation, PyMongo, transactions

Contexte professionnel

Vous rejoignez l'équipe *data* d'une plateforme de streaming type « **MFlix** ». On vous confie le jeu de données officiel `sample_mflix` de MongoDB — des données **réelles** issues d'IMDB : **23 539 films** (`movies`) et **50 304 commentaires** d'utilisateurs (`comments`). Les deux collections sont **liées par référence** : chaque commentaire porte un `movie_id` (ObjectId) censé pointer vers un film.

Votre mission du jour : **décider du modèle (imbriquer vs référencer)**, **diagnostiquer l'intégrité des références**, **indexer intelligemment**, **mesurer les gains avec `explain()`**, **produire des analyses par agrégation**, puis **connecter une application Python (PyMongo)** qui applique deux *patterns* de modélisation et une transaction ACID.

Données réelles = données imparfaites. Ce jeu contient des **années au mauvais type**, des **notes IMDB vides**, un champ pré-calculé **désynchronisé**, et des **milliers de commentaires orphelins**. Vous devez les *détecter par la requête*. Toute réponse chiffrée est **vérifiable** : rendez la commande exacte et le résultat exact.

Objectifs pédagogiques (rappel du cours)

- Appliquer la règle d'or « *data that is accessed together should be stored together* » et **choisir embed vs reference**.
- Reconnaître et appliquer les **patterns** : Computed, Subset, Bucket, Extended Reference.
- Comprendre que **MongoDB n'impose PAS l'intégrité référentielle** (pas de clé étrangère) et en tirer les conséquences.
- Créer les bons **index** (simple, multi-clés, composé — **règle ESR**, text) et **supprimer les inutiles**.
- Lire un plan d'exécution `.explain("executionStats")` : **COLLSCAN vs IXSCAN vs FETCH**, `totalDocsExamined`, `nReturned`.
- Écrire des **agrégations** (`$match`, `$unwind`, `$group`, `$sort`, `$lookup`).
- Se connecter avec **PyMongo** et gérer une **transaction ACID** multi-documents.

Pré-requis techniques

- L'instance MongoDB du **Jour 1** (Docker, `docker compose up -d`). **Attention** : les transactions exigent un **replica set** (voir Partie 5).
- **Python 3.10+**, `pip`, un venv. Installez le driver : `pip install "pymongo>=4.6"`.

- `mongosh` (via `docker exec`) et MongoDB Compass.

Livrables (Teams, avant 17h00)

1. `reponses_jour2.md` — commande + résultat exact pour **chaque question** Q*, et les réponses rédigées R1→R4.
2. `analyses.js` — les agrégations de la Partie 3.
3. `patterns.py` — le script PyMongo (Partie 4) : recalcul du Computed Pattern + Subset Pattern.
4. `transaction.js` (ou `.py`) — la transaction de la Partie 5.
5. `index_bench.md` — tableau `explain()` *avant* / *après* index (Partie 2).

Partie 0 — Import des données réelles (≈ 20 min)

Téléchargez les deux fichiers (format **Extended JSON**, géré nativement par `mongoimport`) :

```
curl -L -o movies.json https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_mflix/movies.json
curl -L -o comments.json https://raw.githubusercontent.com/neelabalan/mongodb-sample-dataset/main/sample_mflix/comments.json
wc -l movies.json comments.json # attendu : 23539 et 50304
```

Importez-les dans la base `mflix` :

```
docker cp movies.json mongo-ipssi:/tmp/movies.json
docker cp comments.json mongo-ipssi:/tmp/comments.json
docker exec mongo-ipssi mongoimport -u admin -p ipssi2025 --authenticationDatabase admin \
  --db mflix --collection movies --drop --file /tmp/movies.json
docker exec mongo-ipssi mongoimport -u admin -p ipssi2025 --authenticationDatabase admin \
  --db mflix --collection comments --drop --file /tmp/comments.json
```

Contrôle P0 : `db.movies.countDocuments({})` = 23539, `db.comments.countDocuments({})` = 50304.
Observez un document de chaque collection : repérez dans `movies` les tableaux (`genres`, `cast`, `directors`), les sous-documents (`imdb`, `tomatoes`, `awards`), et dans `comments` le champ `movie_id` (la référence).

Partie 1 — Modélisation & intégrité référentielle (≈ 50 min)

Ici on **raisonne** le modèle, puis on **prouve** ses limites par la requête.

- **Q1.** Combien de films ? Combien de commentaires ? Combien de **genres distincts** (`distinct("genres")`) ?

- **Q2.** La relation `comments → movies` est modélisée en **référence** (`movie_id`). MongoDB **n'impose aucune contrainte de clé étrangère**. Écrivez une **agrégation** qui compte les **commentaires orphelins** = ceux dont le `movie_id` ne correspond à **aucun** film de `movies`. (indice : `$lookup` de `comments` vers `movies` sur `movie_id = _id`, puis `$match` sur le résultat de jointure vide.) Combien y en a-t-il ?
- **Q3.** Combien de **films distincts** sont référencés par au moins un commentaire ? (agrégation sur `comments` : `$group` par `movie_id`, puis compter les groupes.)
- **Q4. Computed Pattern — première question d'écart.** Le champ `num_mflix_comments` d'un film est un compteur **pré-calculé** (pattern *Computed*).
 - (a) Combien de films **portent** ce champ (`$exists`) ? Rapportez ce nombre au total de films : quel **pourcentage** ?
 - (b) Prenez le film « *The Taking of Pelham 1 2 3* ». Relevez son `num_mflix_comments`, puis comptez le **nombre réel** de commentaires le référençant.
 - (c) **Donnez l'écart absolu et en pourcentage.** Le compteur sur-estime-t-il ou sous-estime-t-il ?
 - (d) Un site affiche « *N* commentaires » sous chaque film à partir de ce champ. Décrivez ce que voit un utilisateur qui clique. Que révèle cet écart sur les compteurs dénormalisés en général ? (Vous mesurerez l'ampleur du problème sur tout le catalogue en Q16.)
- **Q5. Data quality (type bracketing).** Combien de films ont un `year` stocké comme **chaîne** et non comme entier ? (indice : `$type: "string"`) Pourquoi une requête `{ year: { $gte: 2000 } }` **ignore-t-elle silencieusement** ces documents ?
- **Q6.** Combien de films ont une `imdb.rating` **égale à la chaîne vide** `""` ? En quoi est-ce un piège pour un calcul de moyenne ?

Rappel cours — Embed quand 1:1 / 1:peu, lues ensemble, taille bornée < 16 Mo. Reference quand 1:beaucoup / n:m, entités indépendantes, écritures fréquentes.

Partie 2 — Indexation & `explain()` (≈ 55 min)

Pour chaque question, remplissez `index_bench.md` avec `stage`, `totalDocsExamined`, `nReturned` **avant** et **après** l'index.

- **Q7. Index multi-clés.** Requête `db.movies.find({ genres: "Film-Noir" })`.
 - (a) `explain("executionStats")` **avant** tout index : quel `stage` ? Combien de `totalDocsExamined` ? Combien de résultats (`nReturned`) ?
 - (b) Créez l'index adapté sur `genres`. Relancez l'`explain`. Quel `stage` maintenant ? Que devient `totalDocsExamined` ?
- **Q8. Index composé & règle ESR.** On veut : les films de genre **Drama**, sortis **depuis 2000** (`year >= 2000`), **triés** par `imdb.rating` **décroissant**.
 - (a) Donnez d'abord le **nombre** de films correspondant au filtre `{ genres: "Drama", year: { $gte: 2000 } }`.

- (b) D'après la **règle ESR (Equality, Sort, Range)**, dans **quel ordre** placer les 3 champs `genres`, `imdb.rating`, `year` dans l'index composé ? Créez-le et justifiez l'ordre.
- (c) Vérifiez avec `explain` que le tri est **couvert par l'index** (pas d'étape `SORT` en mémoire).
- **Q9. Index text — la troisième solution du Jour 1.** Au Jour 1 (Q9e), vous aviez conclu qu'un `$regex` ne pouvait pas servir une vraie recherche. Construisez maintenant l'alternative.
 - **(a)** Combien de films dont le **titre contient** exactement `Godfather` ?
(`{ title: { $regex: /Godfather/ } }`)
 - **(b)** Créez un index `text` sur `title` et `plot`, puis faites une recherche `$text` sur « **godfather** ». Combien de résultats ?
 - **(c) Donnez l'écart.** Affichez 3 films que seule la version (b) trouve : **pourquoi** sortent-ils ? (*Regardez leur `title` — puis leur `plot`.*)
 - **(d)** `$text` applique **minuscules + découpage en mots + stemming**. Vérifiez le dernier point : cherchez `$text` sur « **godfathers** » (au pluriel). Obtenez-vous le même nombre qu'en (b) ? Qu'en déduisez-vous, et qu'aurait donné un `$regex` ?
 - **(e)** Citez **un cas** où le `$regex` reste préférable au `$text`. (*Indice : que se passe-t-il si l'on cherche une sous-chaîne qui n'est pas un mot entier, comme un numéro de série ou un fragment de code ?*)
- **Q10.** Listez tous les index de `movies` (`getIndexes()`). Lequel n'avez-vous **pas** créé (il existe toujours) ? Puis **supprimez** l'index `text`. Pourquoi un index inutilisé est-il un *coût pur* (cf. cours) ?

Partie 3 — Agrégation analytique (≈ 45 min) — fichier `analyses.js`

Écrivez ces pipelines (`db.movies.aggregate([...])` / `db.comments.aggregate([...])`).

- **Q11. Top 5 des genres** par nombre de films. (`$unwind` sur `genres`, `$group`, `$sort`, `$limit` .)
- **Q12. Nombre de films par décennie** (1990, 2000, 2010...), trié décroissant. (*calcul de la décennie : `$subtract` / `$mod`, ne conserver que les `year` numériques via `$match { year: { $type: "int" } }`.*)
Donnez le **top 3**.
- **Q13. Note IMDB moyenne des films *Drama*** (n'inclure que les notes numériques). Donnez la moyenne à 4 décimales et le nombre de films comptés.
- **Q14. Top 3 réalisateurs** par nombre de films. (`$unwind` sur `directors` .)
- **Q15. `$lookup` inversé. Top 5 des films les plus commentés** : partez de `comments`, `$group` par `movie_id`, `$sort` décroissant, `$limit 5`, puis `$lookup` vers `movies` pour afficher le **titre**. Donnez les 5 titres et leur nombre de commentaires.

Partie 4 — Drivers : PyMongo (≈ 55 min) — fichier `patterns.py`

Installez : `pip install "pymongo>=4.6"`. Connectez-vous **une seule fois** (un `MongoClient` global) :

```
from pymongo import MongoClient
client = MongoClient("mongodb://admin:ipssi2025@localhost:27017/?authSource=admin")
db = client["mflix"]
```

- **Q16. Computed Pattern — réconciliation.** Écrivez une fonction qui, pour **chaque film**, calcule le **vrai** nombre de commentaires le référençant (depuis `comments`) et le compare à `num_mflix_comments`. **Combien de films ont un compteur incohérent ?** (Astuce perf : ne bouclez pas 23 539 fois avec un `count` — faites un `aggregate` `$group` sur `comments`, chargez le résultat dans un `dict` Python, puis comparez.)
- **Q17.** Toujours en PyMongo, **corrigez** le compteur : mettez à jour `num_mflix_comments` avec la vraie valeur (utilisez `bulk_write` avec des `UpdateOne`). Combien de documents `modifiedCount` ? Re-vérifiez que Q16 renvoie désormais **0** incohérence.
- **Q18. Subset Pattern.** Pour les **10 films les plus commentés**, embarquez dans le film un champ `recent_comments` contenant les **3 commentaires les plus récents** (par `date` décroissante), en ne gardant que `{ name, text, date }`. Vérifiez sur un film que le tableau contient bien 3 sous-documents. (Pourquoi n'embarque-t-on que 3 commentaires et pas les 161 ?)

Partie 5 — Transaction ACID multi-documents (≈ 20 min)

Attention — les transactions nécessitent un **replica set**. Démarrez une instance dédiée :

```
docker run -d --name mongo-rs -p 27018:27017 mongo:7.0 --replSet rs0
docker exec mongo-rs mongosh --port 27017 --eval "rc = rs.initiate()"
```

Réimportez-y `movies` et `comments`, puis connectez-vous sur `localhost:27018`.

- **Q19. Scénario modération** : un commentaire doit être supprimé **et** le compteur `num_mflix_comments` du film décrémenté de 1 — **atomiquement**. Écrivez la transaction (mongosh `startSession()/startTransaction()/commitTransaction()`, ou PyMongo `with client.start_session() as s: s.with_transaction(...)`). Vérifiez qu'en cas d'erreur au milieu, **rien** n'est appliqué (testez un `abortTransaction`). Que garantit chacune des lettres **A-C-I-D** ici ?

Partie 6 — Réflexion (≈ 15 min) — dans `reponses_jour2.md`

Règle de correction : chaque réponse doit citer **au moins un chiffre que vous avez vous-même relevé**, avec le numéro de la question d'origine. Une réponse générale sans aucun de vos résultats ne vaut pas de points. R3 exige en plus une manipulation.

- **R1. Ce que le SGBD ne fait plus pour vous.** Au vu de la Partie 1, **quelle responsabilité** passe du SGBD vers l'application ? Chiffrez le problème : rapportez le nombre d'orphelins (Q2) au nombre total de commentaires — quel **pourcentage** de votre table de commentaires pointe dans le vide ? Citez ensuite **2 stratégies** pour l'éviter côté application, et pour chacune, dites **ce qu'elle coûte** (performance, complexité, ou couverture partielle).
- **R2. Embed vs reference — la borne.** Pour la relation film ↔ commentaires, justifiez pourquoi **référencer** est le bon choix ici. Appuyez-vous sur **votre Q15** : combien de commentaires porte le film le plus commenté ? Reprenez ensuite la méthode de calcul du **Jour 1 (R3)** — estimez le poids d'un commentaire et déduisez la taille qu'atteindrait un film si l'on imbriquait tout. Est-ce la limite des 16 Mo qui tranche, ou autre chose ? Enfin : dans quel cas précis **imbriquerait-on** quand même ?
- **R3. ESR — vérifié par l'expérience.** Réexpliquez avec **vos mots** pourquoi l'ordre *Equality* → *Sort* → *Range* est optimal. Puis **prouvez-le** : créez un second index composé avec les champs dans le **mauvais** ordre (`{ genres: 1, year: 1, "imdb.rating": -1 }`), forcez son utilisation avec `.hint()`, et relancez l'`explain` de la Q8.
 - **(a)** Un stage `SORT` apparaît-il ? Relevez `totalKeysExamined` et `totalDocsExamined` pour les **deux** index.
 - **(b)** **Donnez l'écart** entre les deux. Lequel est le plus coûteux, et de combien ?
 - **(c)** Que se passe-t-il si le tri en mémoire dépasse les **32 Mo** autorisés par défaut ? (Cherchez `allowDiskUse` et l'erreur associée.)
- **R4. Patterns — le bénéfice et sa facture.** Le champ `num_mflix_comments` illustre le **Computed Pattern**. Donnez **un bénéfice** et **un risque**, chacun chiffré à partir de vos mesures : le bénéfice en vous demandant ce que coûterait un recomptage à chaque affichage (Q3 : combien de films sont concernés ?), le risque à partir de la Q16 (**combien** de compteurs étaient faux, sur combien de films portant le champ — donnez le **pourcentage**). Concluez : à quelle condition ce pattern est-il acceptable en production ?

Pour aller plus loin (facultatif)

- **B1. Covered query.** Construisez une requête entièrement **couverte** par un index (résultat lu **uniquement** dans l'index, `totalDocsExamined = 0`, stage **PROJECTION_COVERED**, pas de `FETCH`). Montrez l'`explain`.
 - **B2. Index partiel.** Créez un index **partiel** ne portant que sur les films `{ type: "series" }` (254 docs) et prouvez le gain mémoire vs un index complet.
 - **B3. TTL.** Créez une collection `sessions` avec un index **TTL** de 3600 s sur un champ `createdAt`. Expliquez le cas d'usage.
-

Checklist finale

`reponses_jour2.md` (Q1→Q19 + R1→R4)

analyses.js

patterns.py

transaction.js

index_bench.md